Java Collection Framework: Important methods by Interface

1 . Title: Demonstration of Important Methods of Collection Interface in Java

Aim / Objective : To write and execute a Java program to demonstrate the important methods of the “Collection” interface such as “add()”, “addAll()”, “remove()”, “removeAll()”, “contains()”, “containsAll()”, “size()”, “isEmpty()”, “clear()”, and “iterator()”.

System Requirements :

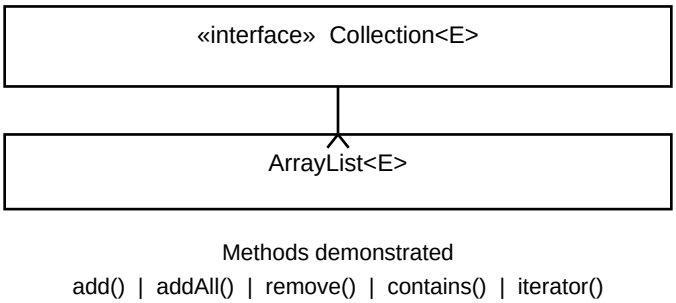
Hardware Requirements : Computer/Laptop , Minimum 4 GB RAM ,Keyboard and Mouse

Software Requirements : Java Development Kit (JDK) , Java compiler , Java Runtime Environment (JRE) , VS Code / Eclipse / IntelliJ IDEA / any Java-supported IDE, Operating System: Windows/Linux/macOS

Algorithm :

1. Start the program.
2. Import the “java.util” package.
3. Create two “Collection” objects using “ArrayList”.
4. Add elements using the “add()” method.
5. Add all elements of the second collection using “addAll()”.
6. Check whether an element exists using “contains()”.
7. Check whether all elements of another collection exist using “containsAll()”.
8. Find the number of elements using “size()”.
9. Check whether the collection is empty using “isEmpty()”.
10. Remove an element using “remove()”.
11. Remove all elements belonging to another collection using “removeAll()”.
12. Traverse the collection using “iterator()”.
13. Remove all remaining elements using “clear()”.
14. Stop the program.

UML Diagram



|. |

Collection Collection

C c2

Main methods used

Collection

|

+-- add()

+-- addAll()

+-- remove()

+-- removeAll()

+-- contains()

+-- containsAll()

+-- size()

+-- isEmpty()

+-- clear()

+-- iterator()

Source Code :

```
import java.util.ArrayList;
 import java.util.Collection;
 public class CollectionDemo {
public static void main(String[] args) {
        Collection<Integer> c1 = new ArrayList<>();
        Collection<Integer> c2 = new ArrayList<>();
  // add()
        c1.add(10);
        c1.add(20);
        c1.add(30)
        c2.add(20);
        c2.add(30);
        System.out.println(“Collection 1: “ + c1);
        System.out.println(“Collection 2: “ + c2);
        // addAll()
       c1.addAll(c2);
        System.out.println(“After addAll(): “ + c1);
        // contains()
        System.out.println(“Contains 20: “ + c1.contains(20));
        // containsAll()
        System.out.println(“Contains all c2: “ + c1.containsAll(c2));
        // size()
        System.out.println(“Size: “ + c1.size());
        // isEmpty()
        System.out.println(“Is Empty: “ + c1.isEmpty());
        // remove()
       c1.remove(10);
```

```
        System.out.println("After remove(10): " + c1);
        // removeAll()
        c1.removeAll(c2);
        System.out.println("After removeAll(c2): " + c1);
        // Add elements again
        c1.add(50);
        c1.add(60);
        // iterator()
        System.out.print("Using Iterator: ");
        Iterator<Integer> itr = c1.iterator();
        while (itr.hasNext()) {
            System.out.print(itr.next() + " ");
        }
        System.out.println();
        // clear()
        c1.clear();
        System.out.println("After clear(): " + c1);
    }
}
```

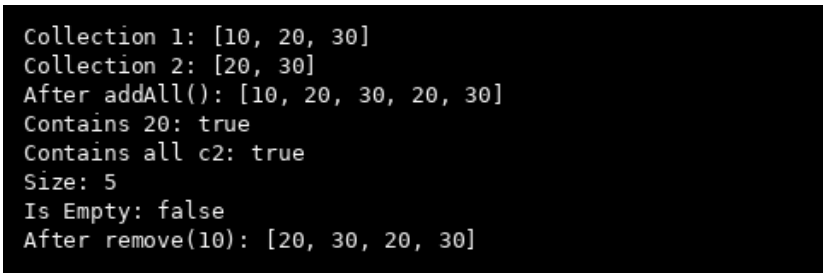
Execution and Compilation Commands :

Save the file as: CollectionDemo.java

Compile the program : Javac CollectionDemo.java

Execute the program: Java CollectionDemo

Output Screenshot :



Result : Thus, the Java program was successfully executed and the important methods of the “Collection” interface were demonstrated.

2. Title. : Demonstration of Important Methods of List Interface in Java

Aim / Objective : To write and execute a Java program to demonstrate the important methods of the “List” interface.

System Requirements :

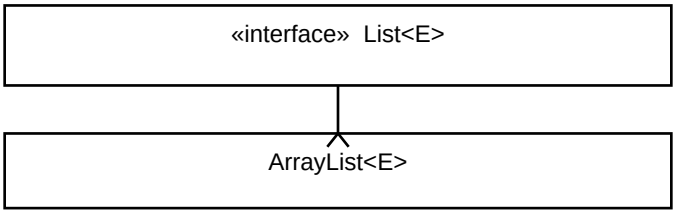
Hardware: Computer/Laptop, minimum 4 GB RAM, Keyboard and Mouse.

Software: JDK, JRE, Java-supported IDE such as VS Code/Eclipse, and Windows/Linux/macOS.

Algorithm :

1. Start the program.
2. Create an “ArrayList”.
3. Add elements using “add()”.
4. Insert an element at a specified position using “add(index, element)”.
5. Retrieve an element using “get()”.
6. Replace an element using “set()”.
7. Remove an element using “remove()”.
8. Find the first and last occurrence using “indexOf()” and “lastIndexOf()”.
9. Create a portion of the list using “subList()”.
10. Sort the list using “sort()”.
11. Stop.

UML Diagram



```
            |
        ArrayList<E>
            |
     |--------------|
   add()            get()
   set()            remove()
   sort()           subList()
```

Source Code :

```
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;
public class ListDemo {
   public static void main(String[] args) {
          List<Integer> list = new ArrayList<>();
      // Add elements
       list.add(30);
       list.add(10);
       list.add(20);
       list.add(10);
       System.out.println(“List: “ + list);
      // Add element at index
        list.add(1, 15);
       System.out.println(“After add(1,15): “ + list);
       System.out.println(“Element at index 2: “ + list.get(2));
     // Retrive element
       list.set(2, 25);
       System.out.println(“After set(): “ + list);
```

```
    // Remove element at index
        list.remove(1);
        System.out.println("After remove(): " + list);
        System.out.println("First index of 10: " + list.indexOf(10));
        System.out.println("Last index of 10: " + list.lastIndexOf(10));
        System.out.println("SubList: " + list.subList(1, 3));
        list.sort(Comparator.naturalOrder());
        System.out.println("After sort(): " + list);
    }
}
```

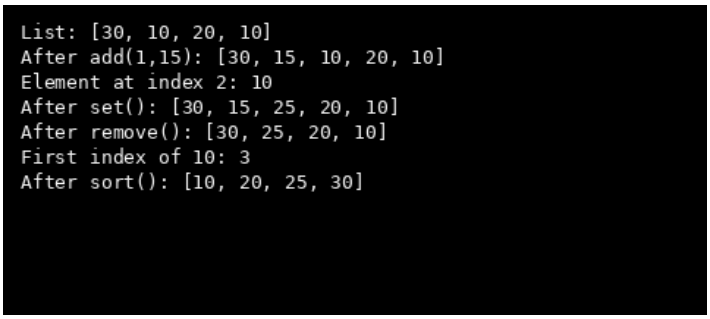
Execution and Compilation Commands:

Save as : ListDemo

Compile as : Javac ListDemo.java

Execute as: Java ListDemo

Output Screenshot :



Result :

Thus, the important methods of the "List" interface were successfully demonstrated.

3. Title : Demonstration of Important Methods of Set Interface in Java

Aim / Objective : To write and execute a Java program to demonstrate the important methods of the "Set" interface.

System Requirements :

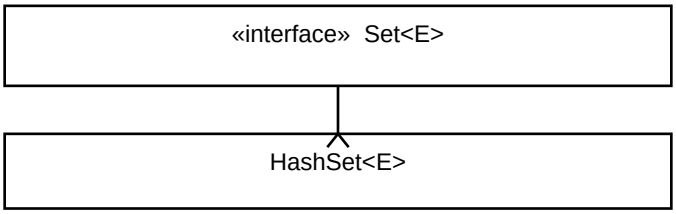
Hardware: Computer/Laptop, minimum 4 GB RAM.

Software: JDK, JRE, Java-supported IDE, and suitable operating system.

Algorithm

1. Start.
2. Create a “HashSet”.
3. Add elements using “add()”.
4. Add a duplicate element and observe that it is not stored again.
5. Check an element using “contains()”.
6. Find the size using “size()”.
7. Check whether the set is empty.
8. Remove an element.
9. Clear the set.
10. Stop.

UML Diagram :



Methods demonstrated

add() | contains() | size() | isEmpty() | remove() | clear()

| |. |. |

Add remove contains size

Source Code

```
import java.util.HashSet;
```

```
import java.util.Set;
class SetDemo {
    public static void main(String[] args) {
         Set<Integer> set = new HashSet<>();
     // Add elements
        set.add(10);
        set.add(20);
        set.add(30);
        set.add(20);
        System.out.println(“Set: “ + set);
      // Contains ()
        System.out.println(“Contains 20: “ + set.contains(20));
       // size()
        System.out.println(“Size: “ + set.size());
        System.out.println(“Is Empty: “ + set.isEmpty());
       // Remove()
        set.remove(10);
        System.out.println(“After remove(10): “ + set);
       // Clear()
        set.clear();
        System.out.println(“After clear(): “ + set);
    }
}
```

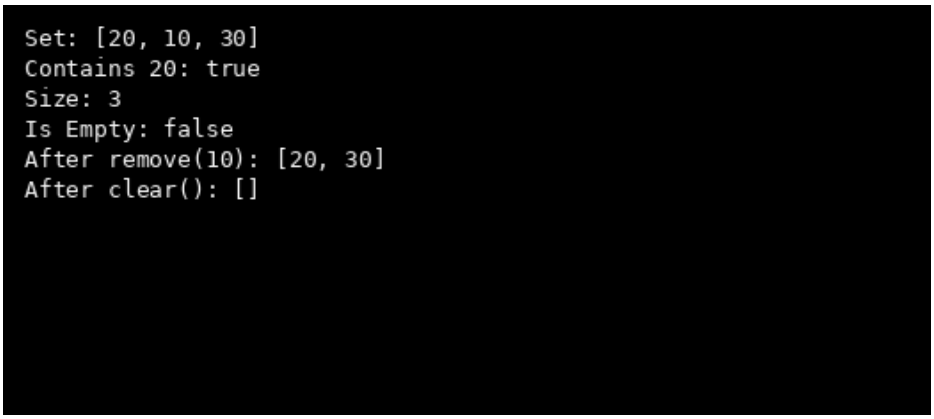
Execution and Compilation Commands :

Save as : SetDemo.java

Compilation: Javac SetDemo.java

Execution : Java SetDemo

Output Screenshot :



Result :

Thus, the important methods of the “Set” interface were successfully demonstrated.

4. Title : Demonstration of Important Methods of SortedSet Interface in Java

Aim / Objective : To write and execute a Java program to demonstrate the important methods of the “SortedSet” interface.

System Requirements :

Hardware: Computer/Laptop, minimum 4 GB RAM.

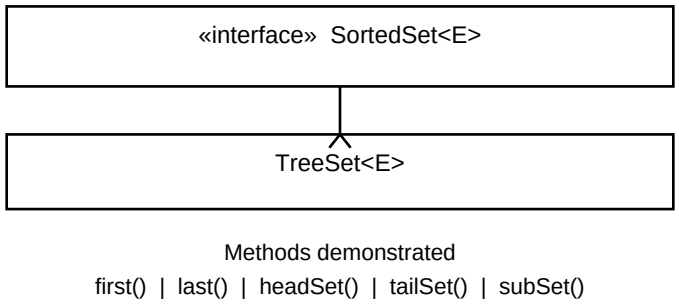
Software: JDK, JRE, Java IDE, and operating system.

Algorithm:

1. Start.
2. Create a “TreeSet”.
3. Add several unsorted elements.
4. Display the automatically sorted set.
5. Find the first and last elements.
6. Display “headSet()”, “tailSet()”, and “subSet()”.
7. Display the comparator.

8. Stop.

UML Diagram :



|

TreeSet<E>

Source Code

```
import java.util.SortedSet;
import java.util.TreeSet;
public class SortedSetDemo {
    public static void main(String[] args) {
        SortedSet<Integer> set = new TreeSet<>();
        // Add elements in sorted
        set.add(40);
        set.add(10);
        set.add(30);
        set.add(20);
        System.out.println(“Sorted Set: “ + set);
          // first()
        System.out.println(“First: “ + set.first());
        System.out.println(“Last: “ + set.last());
```

```
        // headset ()
        System.out.println("HeadSet(30): " +set.headSet(30));
        System.out.println("TailSet(20): " + set.tailSet(20));
        System.out.println("SubSet(20,40): " + set.subSet(20, 40));
        // comparator()
        System.out.println("Comparator: " + set.comparator());
    }
}
```

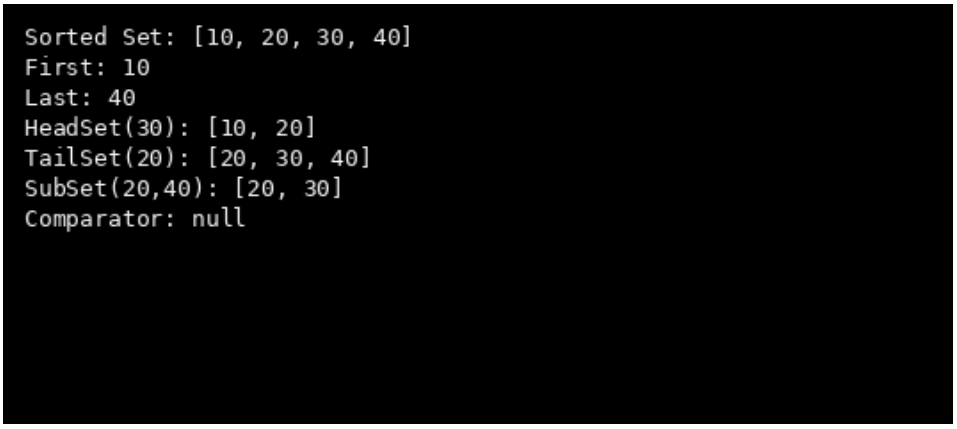
Execution and Compilation Commands :

Save as : SortedSetDemo.java

Compile as : Javac SortedSetDemo.java

Execute as : Java SortedSetDemo

Output Screenshot :



Result : Thus, the important methods of the "SortedSet" interface were successfully demonstrated.

5.Title :. Demonstration of Important Methods of NavigableSet Interface in Java.

Aim / Objective :. To demonstrate navigation and removal methods of the "NavigableSet" interface.

System Requirements :

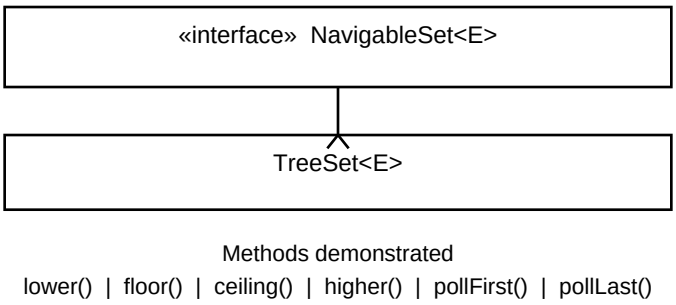
Hardware: Computer/Laptop, minimum 4 GB RAM.

Software: JDK, JRE, Java IDE, and operating system.

Algorithm

1. Start.
2. Create a TreeSet as a NavigableSet.
3. Add sorted elements.
4. Find lower, floor, ceiling, and higher elements.
5. Remove the first and last elements using pollFirst() and pollLast().
6. Display the descending view.
7. Stop.

UML Diagram :



Source Code :

```
import java.util.NavigableSet;
import java.util.TreeSet;
public class NavigableSetDemo {
    public static void main(String[] args) {
        NavigableSet<Integer> set = new TreeSet<>();
         // Add elements in sorted way
        Set.add(10);
        Set.add(20);
        Set.add(30);
```

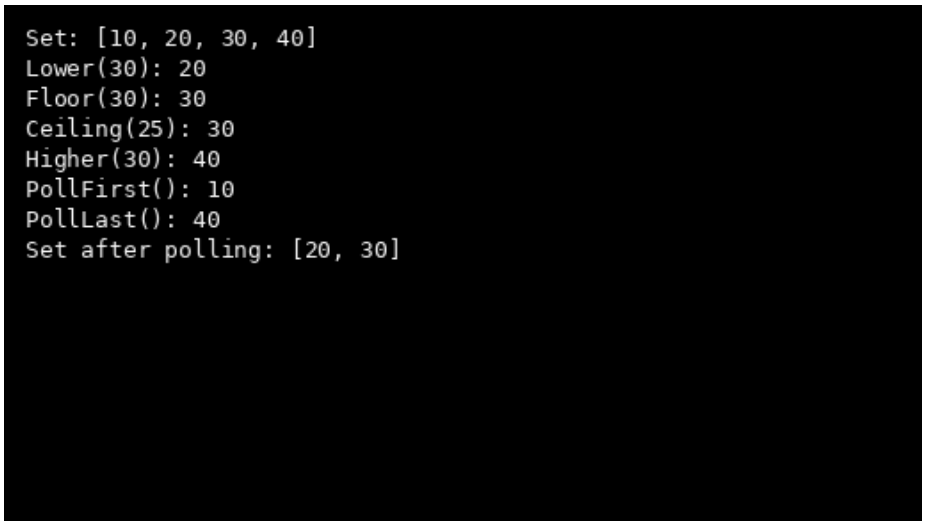
```
        Set.add(40);
        System.out.println("Set: " + set);
        // lower() , floor() , ceiling() , higher()
        System.out.println("Lower(30): " + set.lower(30));
        System.out.println("Floor(30): " + set.floor(30));
        System.out.println("Ceiling(25): " + set.ceiling(25));
        System.out.println("Higher(30): " + set.higher(30));
        // Remove first and last elements
        System.out.println("PollFirst(): " + set.pollFirst());
        System.out.println("PollLast(): " + set.pollLast());
        System.out.println("Set after polling: " + set);
        System.out.println("Descending Set: " + .
set.descendingSet());
    }
}
```

Execution and Compilation Commands :

Javac NavigableSetDemo.java

Java NavigableSetDemo

Output Screenshot:



Result :

Thus, the important methods of the NavigableSet interface were successfully demonstrated.

6.Title : Demonstration of Important Methods of Queue Interface in Java

Aim / Objective : To demonstrate the insertion, deletion, and retrieval methods of the “Queue” interface.

System Requirements :

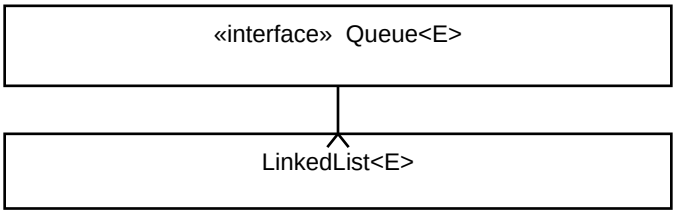
Hardware: Computer/Laptop, minimum 4 GB RAM.

Software: JDK, JRE, Java IDE, and operating system.

Algorithm

1. Start.
2. Create a “Queue” using “LinkedList”.
3. Insert elements using “add()” and “offer()”.
4. Retrieve the head using “element()” and “peek()”.
5. Remove the head using “remove()” and “poll()”.
6. Display the queue.
7. Stop.

UML Diagram :



LinkedList<E>

Source Code :

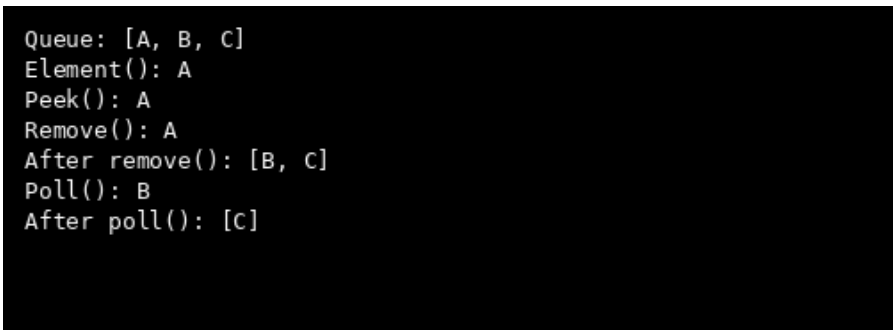
```
import java.util.LinkedList;
import java.util.Queue;
public class QueueDemo {
    public static void main(String[] args) {
        // Create a Queue using LinkedList
        Queue<String> queue = new LinkedList<>();
        queue.add(“A”);
        queue.add(“B”);
        queue.offer(“C”);
        System.out.println(“Queue: “ + queue);
        System.out.println(“Element(): “ + queue.element());
        System.out.println(“Peek(): “ + queue.peek());
        // Remove elements from the front of the queue
        System.out.println(“Remove(): “ + queue.remove());
        System.out.println(“After remove(): “ + queue);
        System.out.println(“Poll(): “ + queue.poll());
        System.out.println(“After poll(): “ + queue);
    }
}
```

Execution and Compilation Commands :

Compile as :Javac QueueDemo.java

Execute as : Java QueueDemo

Output Screenshot :



Result : Thus, the important methods of the “Queue” interface were successfully demonstrated.

7.Title : Demonstration of Important Methods of Deque Interface in Java

Aim / Objective : To demonstrate insertion, deletion, and retrieval operations at both ends of a “Deque”.

System Requirements :

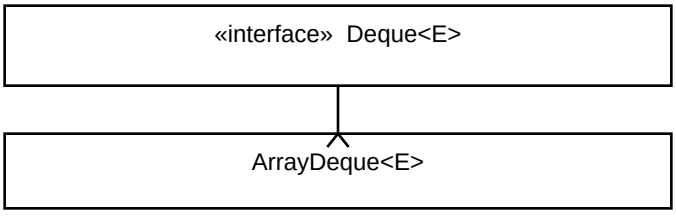
Hardware: Computer/Laptop, minimum 4 GB RAM.

Software: JDK, JRE, Java IDE, and operating system.

Algorithm :

1. Start.
2. Create an “ArrayDeque”.
3. Insert elements at both ends using “addFirst()”, “addLast()”, “offerFirst()”, and “offerLast()”.
4. Retrieve both ends using “peekFirst()” and “peekLast()”.
5. Remove elements using “removeFirst()”, “removeLast()”, “pollFirst()”, and “pollLast()”.
6. Display the deque after each operation.
7. Stop.

UML Diagram :



Methods demonstrated

addFirst() | addLast() | peekFirst() | peekLast() | pollFirst() | pollLast()

Front □------□ Rear

addFirst() addLast()

pollFirst() pollLast()

peekFirst() peekLast()

Source Code :

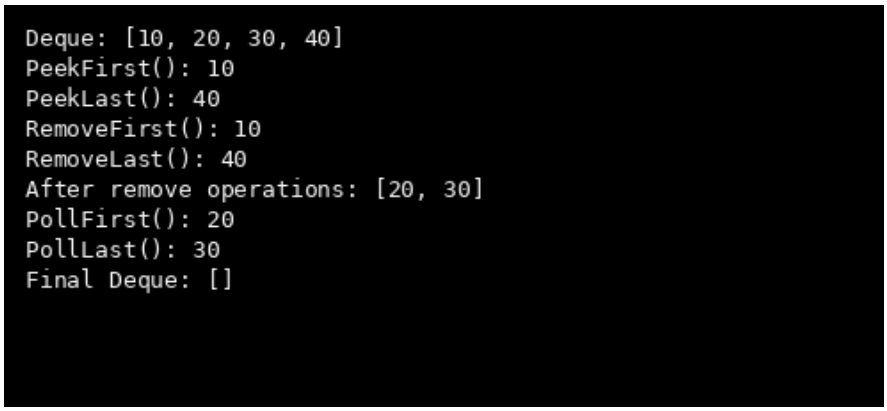
```
import java.util.ArrayDeque;
import java.util.Deque;
public class DequeDemo {
    public static void main(String[] args) {
        // Create a Deque using ArrayDeque
        Deque<Integer> deque = new ArrayDeque<>();
        qeque.addFirst(20);
        qeque.addLast(30);
        qeque.offerFirst(10);
        qeque.offerLast(40);
        System.out.println(“Deque: “ + deque);
        System.out.println(“PeekFirst(): “ + deque.peekFirst());
        System.out.println(“PeekLast(): “ + deque.peekLast());
        // Remove elements from both ends
        System.out.println(“RemoveFirst(): “ + deque.removeFirst());
        System.out.println(“RemoveLast(): “ + deque.removeLast());
        System.out.println(“After remove operations: “ + deque);
        System.out.println(“PollFirst(): “ + deque.pollFirst());
        System.out.println(“PollLast(): “ + deque.pollLast());
        System.out.println(“Final Deque: “ + deque);
    }
}
```

Execution and Compilation Commands

Compile as : Javac DequeDemo.java

Execute as : Java DequeDemo

Output Screenshot :



Result : Thus, the important methods of the “Deque” interface were successfully demonstrated.

8.Title : Demonstration of Important Methods of Map Interface in Java

Aim / Objective : To demonstrate the important methods of the “Map” interface for storing key-value pairs.

System Requirements :

Hardware: Computer/Laptop, minimum 4 GB RAM.

Software: JDK, JRE, Java IDE, and operating system.

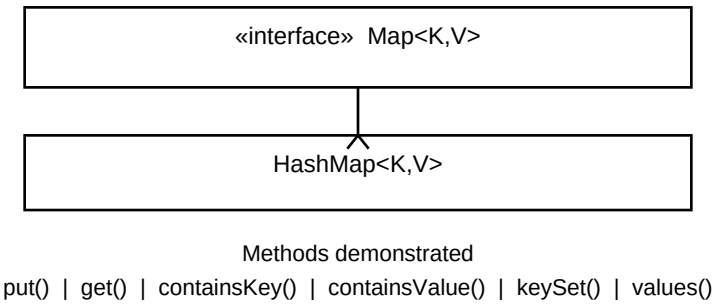
Algorithm :

1. Start.
2. Create a “HashMap”.
3. Add key-value pairs using “put()”.
4. Retrieve a value using “get()”.
5. Check keys and values using “containsKey()” and “containsValue()”.
6. Display keys, values, and entries.
7. Find the size and check whether the map is empty.
8. Remove a mapping using “remove()”.

9. Clear the map.

10. Stop.

UML Diagram



|. |. |

Key Value Entry

Source Code :

```
import java.util.HashMap;
import java.util.Map;
public class MapDemo {
    public static void main(String[] args) {
        // Create a Map using HashMap
        Map<Integer, String> map = new HashMap<>();
        map.put(101, “Alice”);
        map.put(102, “Bob”);
        map.put(103, “Charlie”);
       System.out.println(“Map: “ + map);
        System.out.println(“Get key 102: “ + map.get(102));
        System.out.println(“Contains key 101: “ + map.containsKey(101));
        System.out.println(“Contains value Bob: “ + map.containsValue(“Bob”));
```

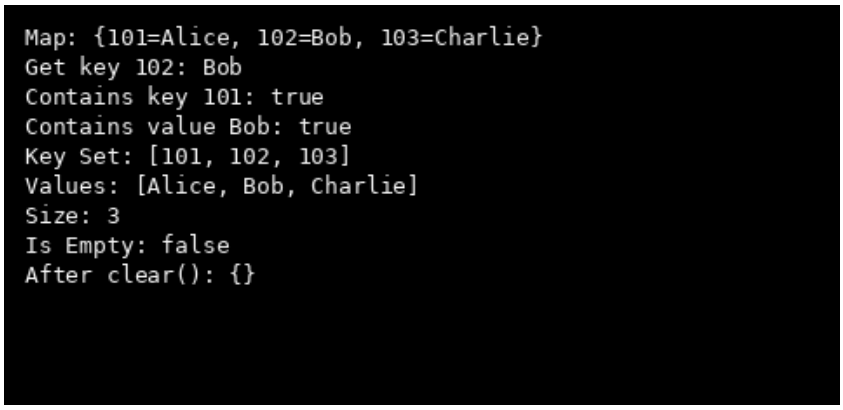
```
        // Display keys, values, and key-value entries
        System.out.println("Key Set: " + map.keySet());
        System.out.println("Values: " + map.values());
        System.out.println("Entry Set: " + map.entrySet());
        System.out.println("Size: " + map.size());
        System.out.println("Is Empty: " + map.isEmpty());
        map.remove(103);
        System.out.println("After remove(103): " + map);
        map.clear();
        System.out.println("After clear(): " + map);
    }
}
```

Execution and Compilation Commands :

Compile as: Javac MapDemo.java

Execute as : Java MapDemo

Output Screenshot :



Result :

Thus, the important methods of the "Map" interface were successfully demonstrated.

9. Title : Demonstration of Important Methods of SortedMap Interface in Java

Aim / Objective : To demonstrate the important methods of the "SortedMap" interface.

System Requirements :

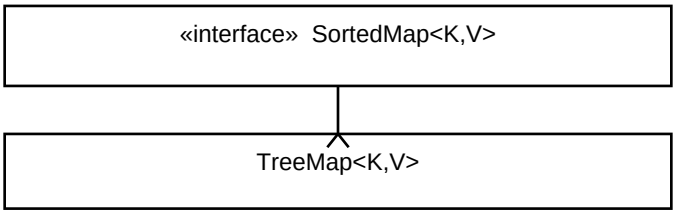
Hardware: Computer/Laptop, minimum 4 GB RAM.

Software: JDK, JRE, Java IDE, and operating system.

Algorithm :

1. Start.
2. Create a “TreeMap”.
3. Insert key-value pairs.
4. Observe that keys are maintained in sorted order.
5. Find the first and last keys.
6. Display “headMap()”, “tailMap()”, and “subMap()”.
7. Display the comparator.
8. Stop.

UML Diagram



Methods demonstrated

firstKey() | lastKey() | headMap() | tailMap() | subMap()

Source Code :

```
import java.util.SortedMap;
import java.util.TreeMap;
public class SortedMapDemo {
    public static void main(String[] args) {
        // TreeMap maintains keys in sorted order
```

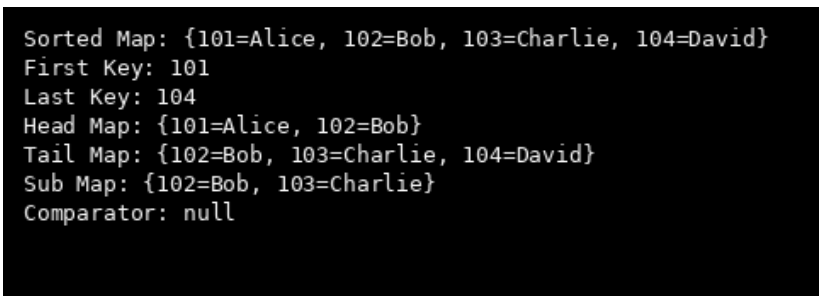
```
        SortedMap<Integer, String> map = new TreeMap<>();
        map.put(103, “Charlie”);
        map.put(101, “Alice”);
        map.put(104, “David”);
        map.put(102, “Bob”);
        System.out.println(“Sorted Map: “ + map);
        System.out.println(“First Key: “ + map.firstKey());
        System.out.println(“Last Key: “ + map.lastKey());
        // Display different portions of the sorted map
        System.out.println(“Head Map: “ + map.headMap(103));
        System.out.println(“Tail Map: “ + map.tailMap(102));
        System.out.println(“Sub Map: “ + map.subMap(102, 104));
        System.out.println(“Comparator: “ + map.comparator());
    }
}
```

Execution and Compilation Commands :

Compile as : Javac SortedMapDemo.java

Execute as : Java SortedMapDemo

Output Screenshot :



Result : Thus, the important methods of the “SortedMap” interface were successfully demonstrated.

10 .Title : Demonstration of Important Methods of NavigableMap Interface in Java

Aim / Objective : To demonstrate the navigation and entry-manipulation methods of the “NavigableMap” interface.

System Requirements :

Hardware: Computer/Laptop, minimum 4 GB RAM.

Software: JDK, JRE, Java IDE, and operating system.

Algorithm :

1. Start.
2. Create a “TreeMap” as a “NavigableMap”.
3. Insert key-value pairs.
4. Find lower, floor, ceiling, and higher keys.
5. Display the first and last entries.
6. Remove the first and last entries.
7. Display the map in descending order.
8. Stop.

UML Diagram

<<interface>>

NavigableMap

|

Extends

|

SortedMap

|

Map

|

TreeMap

Source Code :

```
import java.util.NavigableMap;
import java.util.TreeMap;
import java.util.Map;
public class NavigableMapDemo {
    public static void main(String[] args) {
        // Create a NavigableMap using TreeMap
        NavigableMap<Integer, String> map = new TreeMap<>();
        map.put(10, “A”);
        map.put(20, “B”);
        map.put(30, “C”);
        map.put(40, “D”);
        System.out.println(“Map: “ + map);
        System.out.println(“Lower Key: “ + map.lowerKey(30));
        System.out.println(“Floor Key: “ + map.floorKey(30));
        System.out.println(“Ceiling Key: “ + map.ceilingKey(25));
        System.out.println(“Higher Key: “ + map.higherKey(30));
        System.out.println(“First Entry: “ + map.firstEntry());
        System.out.println(“Last Entry: “ + map.lastEntry());
        // Remove the first and last entries
        System.out.println(“Poll First Entry: “ + map.pollFirstEntry());
        System.out.println(“Poll Last Entry: “ + map.pollLastEntry());
        System.out.println(“Map after polling: “ + map);
        System.out.println(“Descending Map: “ + map.descendingMap());
    }
```

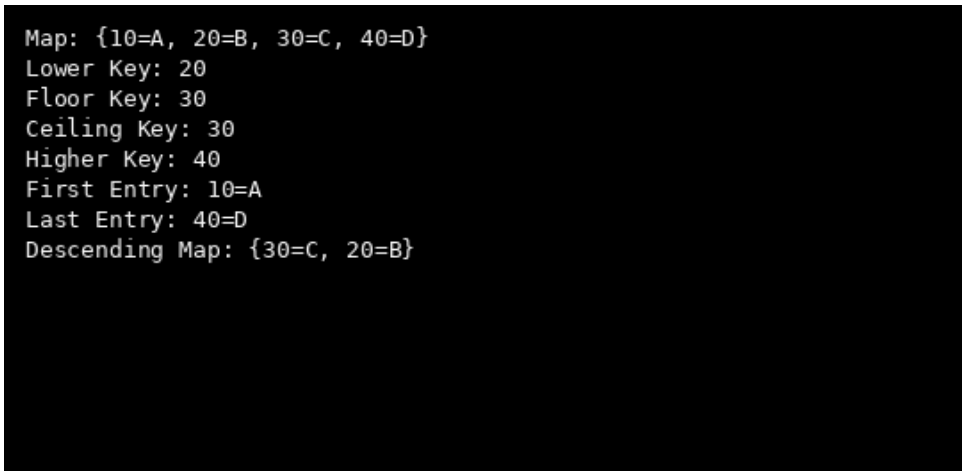
}

Execution and Compilation Commands :

Compile as : Javac NavigableMapDemo.java

Execute as : Java NavigableMapDemo

Output Screenshot :



Result: Thus, the important methods of the “NavigableMap” interface were successfully demonstrated.

11. Title : Demonstration of Important Methods of Iterator Interface in Java

Aim / Objective :. To demonstrate the important methods of the “Iterator” interface for traversing and modifying a collection.

System Requirements :

Hardware: Computer/Laptop, minimum 4 GB RAM.

Software: JDK, JRE, Java IDE, and operating system.

Algorithm :

1. Start.

2. Create an “ArrayList”.

3. Create an “Iterator”.

4. Check for the next element using “hasNext()”.

5. Retrieve the element using “next()”.

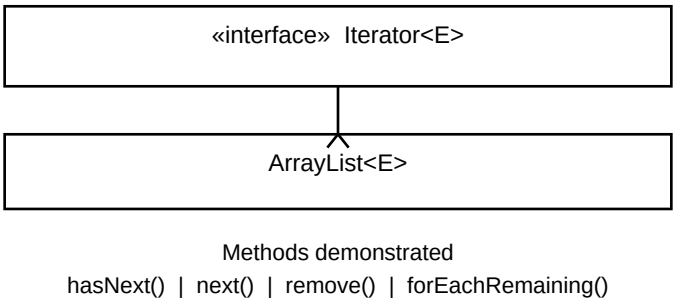
6. Remove an element using “remove()”.

7. Create another iterator.

8. Process remaining elements using “forEachRemaining()”.

9. Stop.

UML Diagram :



Source Code :

```
import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;
public class IteratorDemo {
    public static void main(String[] args) {
        // Create an ArrayList for iteration
        List<Integer> list = new ArrayList<>();
        list.add(10);
        list.add(20);
        list.add(30);
        list.add(40);
        Iterator<Integer> itr = list.iterator();
```

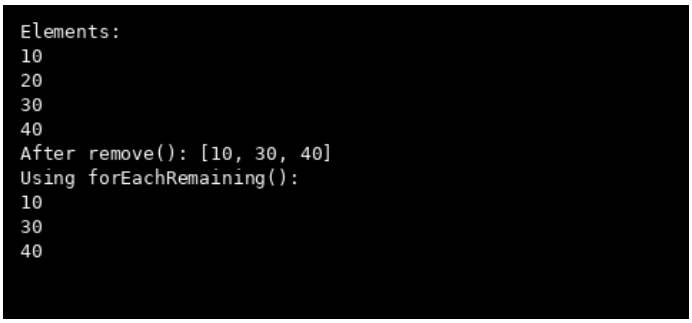
```
        System.out.println("Elements:");
        while (itr.hasNext()) {
            int value = itr.next();
            System.out.println(value);
            // Remove 20 safely using the iterator
            if (value == 20) {
                itr.remove();
            }
        }
        System.out.println("After remove(): " + list);
        System.out.println("Using forEachRemaining():");
        Iterator<Integer> itr2 = list.iterator();
        itr2.forEachRemaining( Value -> System.out.println(value));
    }
}
```

Execution and Compilation Commands :

Compile as : Javac IteratorDemo.java

Execute as : Java IteratorDemo

Output Screenshot :



Result : Thus, the important methods of the “Iterator” interface were successfully demonstrated.

12. Title : Demonstration of Important Methods of ListIterator Interface in Java

Aim / Objective : To demonstrate forward and backward traversal and modification methods of the “ListIterator” interface.

System Requirements :

Hardware: Computer/Laptop, minimum 4 GB RAM.

Software: JDK, JRE, Java IDE, and operating system.

Algorithm :

1. Start.
2. Create an “ArrayList”.
3. Create a “ListIterator”.
4. Traverse forward using “hasNext()” and “next()”.
5. Display “nextIndex()” and “previousIndex()”.
6. Traverse backward using “hasPrevious()” and “previous()”.
7. Modify an element using “set()”.
8. Insert an element using “add()”.
9. Remove an element using “remove()”.
10. Stop.

UML Diagram :

<<interface>>

ListIterator<E>

|

Extends

|

Iterator<E>

|

ArrayList

|

|. |

Forward Backward

Next() previous()

hasNext() hasPrevious()

Source Code :

```
import java.util.ArrayList;
import java.util.List;
import java.util.ListIterator;
public class ListIteratorDemo {
    public static void main(String[] args) {
        // Create a list for forward and backward traversal
        List<Integer> list = new ArrayList<>();
        list.add(10);
        list.add(20);
        list.add(30);
        ListIterator<Integer> itr = list.listIterator();
        System.out.println(“Initial List: “ + list);
        System.out.println(“\nForward Traversal:”);
        while (itr.hasNext()) {
            System.out.println(“Element: “ + itr.next());
            System.out.println(“Next Index: “ + itr.nextIndex());
            System.out.println(“Previous Index: “ + itr.previousIndex());
        }
        System.out.println(“\nBackward Traversal:”);
```

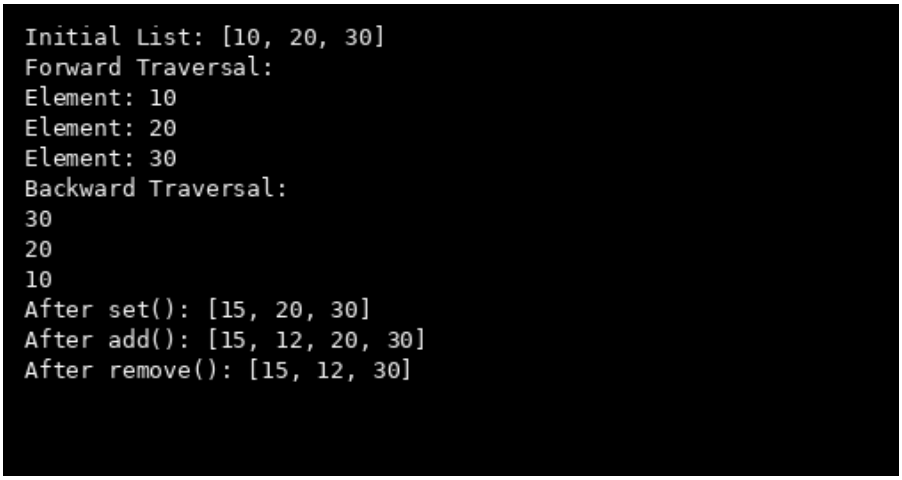
```
        while (itr.hasPrevious()) {
            System.out.println(itr.previous());
        }
        // Modify and insert elements using ListIterator
        itr.next();
        itr.set(15);
        System.out.println("\nAfter set(): " + list);
        itr.add(12);
        System.out.println("After add(): " + list);
        itr.next();
        itr.remove();
        System.out.println("After remove(): " + list);
    }
}
```

Execution and Compilation Commands :

Compile as : Javac ListIteratorDemo.java

Execute as: Java ListIteratorDemo

Output Screenshot :



Result : Thus, the important methods of the “ListIterator” interface were successfully demonstrated.